МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛАБОРАТОРНА РОБОТА №9

Паралельність, багатопроцесність та оптимізація

Варіант 2. Система обліку книг

Дисципліна: Програмування / Python
Тема: Паралельність, багатопроцесність та оптимізація
Варіант: 2 — Система обліку книг
Дата: 26.10.2026

Мета роботи: ознайомитися з основними підходами до паралельного та конкурентного виконання програм у Python, дослідити вплив GIL на CPU-bound та I/O-bound задачі, навчитися використовувати threading, multiprocessing, ThreadPoolExecutor, ProcessPoolExecutor, NumPy та caching; експериментально визначити bottleneck і підтвердити прискорення.

1. Мета та завдання роботи

Основною метою лабораторної роботи є не просто використання засобів паралельності, а експериментальне визначення того, який спосіб оптимізації дійсно прискорює конкретну операцію проєкту.

У результаті виконання роботи необхідно вміти:

• пояснювати різницю між concurrency та parallelism;

• пояснювати принцип роботи GIL;

• визначати CPU-bound та I/O-bound задачі;

• використовувати threading, multiprocessing, ThreadPoolExecutor, ProcessPoolExecutor;

• застосовувати synchronization (Lock) і уникати race conditions;

• вимірювати час виконання (perf_counter / timeit);

• профілювати програму за допомогою cProfile та tracemalloc;

• використовувати NumPy для векторизації;

• використовувати caching з коректною invalidation;

• порівнювати різні реалізації та обчислювати коефіцієнт прискорення.

2. Предметна область (Варіант 2)

Система обліку книг. Кожна книга містить: назву, автора, рік видання та кількість сторінок. Базові операції: додавання книги, пошук за автором, фільтрація за роком, визначення книги з найбільшою кількістю сторінок, обчислення середньої кількості сторінок, статистика авторів.

Для експериментів згенеровано каталог від 50 000 до 150 000 книг. Профілювалися: статистика сторінок, статистика авторів, штучна CPU-bound операція (page_score), паралельне читання файлів (I/O-bound) та кешування статистики автора.

3. Структура проєкту

```
book_manager/
├── pyproject.toml / requirements.txt / README.md
├── src/book_manager/
│   ├── models.py          # dataclass Book
│   ├── analytics.py       # генерація даних, baseline, NumPy, caching
│   ├── parallel.py        # threading, Process/ThreadPool, Lock
│   ├── services.py        # BookCatalog + cache invalidation
│   ├── profiling.py       # cProfile + tracemalloc
│   └── main.py
├── tests/                 # 13 unit-тестів
├── benchmarks/
│   ├── benchmark.py
│   └── results/benchmark_results.csv
└── data/
```

4. Теоретичні відомості (стисло)

4.1. Concurrency vs Parallelism

Concurrency — організація виконання кількох задач так, щоб програма могла перемикатися між ними (ефективно для I/O-bound). Parallelism — фізично одночасне виконання на кількох ядрах CPU (необхідне для прискорення CPU-bound задач).

4.2. GIL (Global Interpreter Lock)

У CPython лише один потік одночасно виконує Python bytecode в межах одного процесу. Тому велика кількість потоків зазвичай не прискорює CPU-bound код. Для важких обчислень доцільніше використовувати multiprocessing (окремі процеси = окремі GIL).

4.3. CPU-bound та I/O-bound

CPU-bound: математичні розрахунки, статистика, image processing. I/O-bound: читання/запис файлів, запити до БД, HTTP. Для I/O-bound добре ThreadPoolExecutor; для CPU-bound — ProcessPoolExecutor або векторизація (NumPy).

5. Реалізація основних компонентів

5.1. Baseline (чисті Python-цикли)

Функції statistics_python, average_pages_python, author_statistics_python реалізовані через звичайні цикли for. Вони служать точкою відліку (baseline) для обчислення speedup.

5.2. NumPy

statistics_numpy та author_statistics_numpy використовують np.asarray + векторизовані mean/min/max/std. Це дозволяє виконувати основні цикли оптимізованим низькорівневим кодом.

5.3. ThreadPoolExecutor / ProcessPoolExecutor

Для CPU-bound експерименту використано штучну функцію page_score (цикл із sin/cos/sqrt). Порівнювалися sequential, ThreadPoolExecutor та ProcessPoolExecutor з різною кількістю workers.

5.4. Caching + invalidation

Використано functools.lru_cache. Ключ кешу містить author + version. При додаванні книги викликається invalidate_author_cache() (збільшується version + cache_clear). Це демонструє коректну cache invalidation.

5.5. Synchronization (Lock)

Продемонстровано race condition при одночасному інкременті спільного лічильника кількома потоками та виправлення через threading.Lock (критична секція).

6. Експериментальні результати

Середовище: Linux, Python 3.12, 2 CPU cores. Усі заміри — середнє з кількох повторів (perf_counter).

6.1. Статистика числового масиву (Python vs NumPy)

Метод	Розмір даних	Середній час, с	Speedup
Python baseline	150 000	0.0174	1.00×
NumPy	150 000	0.0088	1.97×

Висновок: NumPy дає помітне прискорення навіть на відносно простій агрегації завдяки векторизації.

6.2. Статистика авторів

Метод	Розмір даних	Середній час, с	Speedup
Python	80 000	0.0133	1.00×
NumPy	80 000	0.0063	2.11×

6.3. CPU-bound операція (page_score)

Метод	Workers	Середній час, с	Speedup
Sequential (baseline)	1	0.1189	1.00×
ThreadPoolExecutor	4	0.2206	0.54×
ProcessPoolExecutor	4	0.0999	1.19×

ThreadPoolExecutor виявився повільнішим за baseline через GIL та overhead керування потоками. ProcessPoolExecutor дав прискорення (обмежене кількістю ядер — 2 CPU).

6.4. Масштабування за кількістю workers

Workers	ThreadPool, с	ProcessPool, с
1	0.1599	0.1056
2	0.1545	0.0984
4	0.1579	0.1045
8	0.1477	0.0732

ThreadPool майже не залежить від кількості workers (GIL). ProcessPool показує кращі результати при збільшенні workers (доки не перевищено кількість фізичних ядер + overhead).

6.5. Caching

Сценарій	Час, с	Speedup
Cold cache (перший виклик)	0.0031	1.00×
Warm cache (повторний)	~0.0000006	~5000×
Після invalidation	0.0028	≈ cold

Після додавання книги кеш коректно інвалідується і наступний запит знову обчислюється з нуля.

6.6. I/O-bound (паралельне читання файлів)

Метод	Workers	Середній час, с	Speedup
Sequential	1	0.0066	1.00×
ThreadPoolExecutor	8	0.0065	1.03×

На малих локальних файлах overhead потоків нівелює виграш. На реальних мережевих/дискових I/O з більшою затримкою ThreadPoolExecutor зазвичай дає помітне прискорення.

7. Аналіз результатів та bottlenecks

1. Bottleneck CPU-bound Python-коду — GIL. ThreadPoolExecutor не прискорює (навіть сповільнює) важкі обчислення. Рішення: ProcessPoolExecutor або перехід на векторизовані бібліотеки (NumPy).

2. Найкращий результат для числової статистики — NumPy. Векторизація дозволяє виконувати операції в оптимізованому C-коді без накладних витрат інтерпретатора Python.

3. Caching дає величезний виграш при повторних запитах (тисячі разів). Обов'язковою умовою є коректна invalidation після зміни даних.

4. Parallel ≠ automatically faster. Маленький workload, overhead створення процесів, serialization і передача даних можуть зробити паралельну версію повільнішою. Тому вимірювання є обов'язковим.

5. Кількість workers варто підбирати експериментально (1, 2, 4, 8...) і не перевищувати суттєво кількість CPU cores без потреби.

8. Висновки

У ході лабораторної роботи реалізовано повний цикл оптимізації проєкту «Система обліку книг»:

• створено baseline-реалізацію та виміряно її продуктивність;

• виявлено bottleneck за допомогою cProfile / порівняльних замірів;

• застосовано ThreadPoolExecutor, ProcessPoolExecutor, NumPy та caching;

• продемонстровано race condition та виправлено її через Lock;

• реалізовано коректну cache invalidation;

• порівняно щонайменше три реалізації з обчисленням speedup;

• підтверджено, що для CPU-bound задач у CPython ефективнішими є multiprocessing та векторизація, а не threading.

Експериментально підтверджено прискорення: NumPy ≈ 2× на статистиці, ProcessPoolExecutor ≈ 1.2× на CPU-bound (обмежено 2 ядрами), warm cache — тисячі разів. ThreadPoolExecutor на CPU-bound задачі прискорення не дав (вплив GIL).

9. Інструкція з запуску

```
cd book_manager
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Демо
PYTHONPATH=src python -m book_manager.main

# Тести (13 тестів)
PYTHONPATH=src pytest -v

# Повний бенчмарк
PYTHONPATH=src python benchmarks/benchmark.py

# Профілювання
PYTHONPATH=src python -m book_manager.profiling
```

Проєкт готовий до завантаження на GitHub і містить усі необхідні компоненти згідно з вимогами лабораторної роботи №9.